

同一 ObjectKey 的多个副本：预留空间 → 搬运数据 → 完成通知 → 发布 / 释放



按 vLLM MP + CPU 缓存 + 已配置外部后端的普通 eager 模式举例；路径编号对应上方协议图。

读路径由“新请求需要恢复的前缀在哪一层”决定；写路径由“连接器有可保存的新 KV”启动，不必等内存满。

① 由引擎连接器提交; ② 由 CPU 发布完成通知触发; ③ 由请求 lookup / prefetch 触发; ④ 由引擎需要恢复 GPU 页面触发。

A · 首次计算新 prompt，产生可保存的 KV

触发：连接器发现新增、满足完整 chunk 等保存条件的 KV，提交 STORE。

例子: chunk = 256; 首次 1024-token 前缀已算完, 有 4 个完整 chunk 可缓存。

路径: ① GPU → CPU staging → 发布 CPU 副本 → ② 按策略后台写外部。

结果：默认保留 CPU；GPU 原副本不自动删除。不是每个新 token 都单独写出。

前提：保存功能启用且范围未受限；未配置外部后端时，流程止于①。

C · GPU 已无副本，CPU 仍然命中

触发：引擎需要恢复的前缀，在 LMCache CPU 缓存中找到可用副本。

例子：GPU 旧前缀已被回收，但上次①保存的 CPU 副本仍在。

路径: CPU 取得读锁 (C2 → C3) → ④ H2D → GPU 可用 (G2 → G1)。

结果：这部分无需③外部数据读取；保留型 CPU 副本在最后读锁释放后继续驻留。CPU 命中可省外部数据读取，但当前实现仍可能查询外部元数据。

E · 显存吃紧，引擎需要复用旧 GPU 页

触发：推理引擎按自己的页面管理规则，回收不再被使用的 GPU 缓存页。

例子：旧请求结束，已有 CPU / 外部副本；新请求需要腾出 GPU 空间。

路径: G1 → G0; 此前通过① / ②写出的副本, 可在下次按 C / D 恢复。

边界：GPU 回收本身不保证触发①；不能假定回收每一页时都会同步 offload。若存在在途搬运，页面复用须遵守引擎与连接器的完成协议；lazy 模式见下方。

可选 lazy offload: 把“何时启动①”交给延迟写出策略

lazy_offload 默认关闭。开启并使用 EVICTION_AWARE 时，先积累待存对象；预测 GPU 块进入淘汰危险窗口，或配置的最大延迟到期，才提交①。

CPU 副本发布后仍按②后台复制。它不同于 F 的 CPU 水位淘汰；全层未命中时由引擎重算，满足保存条件后再按 A 写出。

示例为满足命中、分配与存取条件的成功主线，不承诺所有模型 / 配置都采用同一保存时机；默认不等于每步或每个 decode token 都写出。

B · 相同前缀仍在 GPU，可直接复用

触发：新请求到来，引擎本地前缀缓存仍有可复用的 GPU 页面。

例子：连续两轮请求复用相同的 system prompt，其 GPU 页尚未被回收。

路径：直接复用 G1；GPU 已覆盖的这部分前缀，不必再通过④恢复。

边界：不代表完全没有跨层 I/O；①补存、③提前预取或后台任务可独立发生。

只讨论可复用的前缀段；新后缀的计算与写出另行判断。

D • GPU / CPU 都没有，外部仍有副本

触发：新请求需要恢复前缀，CPU 未命中，外部 lookup 找到可用副本。

例子：长文档 KV 已写入外部，随后 GPU、CPU 副本都被淘汰；现在再次访问。

路径: ③ 保护外部源 → CPU staging → 发布并取得读锁 → ④ 恢复 GPU。

结果：默认这份新载入的 CPU 对象为 temporary；最后读者结束后即释放。
若使用 retain 预取策略，则保留这份 CPU 副本，以备后续请求复用。

F · CPU 缓存超过淘汰水位

触发：CPU 淘汰控制器检测到占用率超过配置阈值，选择无锁等可淘汰对象。

例子：假设阈值设为 80%，当前占用 90%；选择空闲的冷对象释放空间。

路径: C2 → C0 (DISCARD); 不是此时才触发②, ②由 CPU 写完成独立触发。

结果：若外部已有副本，下次可走 D；若其他层也没有，则缓存 miss、需要重算。
CPU 回收不以“外部一定已有副本”为统一前提，也不会自动删除外部副本。